ESP32 DataLogger CLI - Complete User Manual

Table of Contents

Installation

Requirements

- Python 3.6 or higher
- PySerial library

Install Dependencies

bash

```
pip install pyserial
```

Download Script

Save `datalogger_cli.py` to your computer and make it executable (Linux/Mac):

bash

```
chmod +x datalogger_cli.py
```

Connection Options

These options control how the tool connects to your datalogger device.

`--port <PORT>`

Description: Specify the serial port to connect to.

Usage:


bash

```
# Windows
python datalogger_cli.py --port COM19 --list

# Linux
python datalogger_cli.py --port /dev/ttyUSB0 --list

# Mac
python datalogger_cli.py --port /dev/cu.usbserial-XXX --list
```

When to use:

- When auto-discovery fails
- When you have multiple devices
- For scripts requiring explicit port control

Default: Auto-discovery enabled (no port needed)

`--baud <RATE>`

Description: Set the serial communication baud rate.

Usage:


bash

```
python datalogger_cli.py --port COM19 --baud 115200 --list
```

Default: 115200 (matches firmware default)

Note: Only change if your firmware uses a different baud rate.

`--discover` / `--no-discover`

Description: Enable or disable automatic device discovery.

Usage:


bash

```
# Enable auto-discovery (default)
python datalogger_cli.py --discover --list

# Disable auto-discovery (requires --port)
python datalogger_cli.py --no-discover --port COM19 --list
```

When to use `--no-discover`:

- Multiple serial devices connected
- Faster connection when port is known
- Scripted/automated operations

`--reset`

Description: Toggle DTR/RTS lines to reset the device before connecting.

Usage:


bash

```
python datalogger_cli.py --reset --port COM19 --list
```

When to use:

- Device is unresponsive
- After uploading new firmware
- To force a fresh start

`--no-progress`

Description: Disable progress bar during file downloads.

Usage:


bash

```
python datalogger_cli.py --date 2025-10-07 --no-progress
```

When to use:

- Logging output to files
- Running in scripts/cron jobs
- Cleaner console output

Data Retrieval Commands

`--list`

Description: List all available date logs with file sizes and row counts.

Usage:

bash

```
python datalogger_cli.py --list
```

Example Output:

```
Available logs (5 dates):
--------------------------------------------------
2025-10-01    127 KB    412 rows
2025-10-02    145 KB    485 rows
2025-10-03    138 KB    461 rows
2025-10-04    152 KB    506 rows
2025-10-07     83 KB    276 rows
```

Information Shown:

- **Date:** YYYY-MM-DD format
- **Size:** File size in kilobytes
- **Rows:** Number of data records (excluding header)

When to use:

- Before downloading to see what's available
- Check storage usage
- Verify logging is working

`--date <YYYY-MM-DD>`

Description: Download CSV file for a specific date.

Usage:

bash

```
# Download to default location (./logs/2025-10-07.csv)
python datalogger_cli.py --date 2025-10-07

# Download to specific location
python datalogger_cli.py --date 2025-10-07 --out D:\Data\today.csv

# Download multiple dates
python datalogger_cli.py --date 2025-10-05 --out ./day1.csv
python datalogger_cli.py --date 2025-10-06 --out ./day2.csv
python datalogger_cli.py --date 2025-10-07 --out ./day3.csv
```

Example Output:

```
Downloading 2025-10-07...
Downloading... 100% (84523/84523 bytes)
✓ Saved: ./logs/2025-10-07.csv (84,523 bytes)
```

CSV Format:

csv

```
time,temperature,humidity,vbat_mV
08:00:00,25.34,62.15,3712
08:05:00,25.41,61.98,3710
08:10:00,25.38,62.07,3709
```

Features:

- CRC32 verification (automatic)

- Resume capability (uses `.part` temp file)
- Progress indicator (disable with `--no-progress`)

`--out <PATH>`

Description: Specify output file path for downloaded CSV.

Usage:


bash

```
python datalogger_cli.py --date 2025-10-07 --out ./data/measurement.csv
```

Default: `./logs/<date>.csv`

Tips:

- Directories are created automatically
- Use absolute or relative paths
- Extension should be `.csv`

RTC (Real-Time Clock) Commands

`--time`

Description: Query the current date and time from the RTC.

Usage:


bash

```
python datalogger_cli.py --time
```

Example Output:

```
Current time: 2025-10-07 14:35:22
```

When to use:

- Verify RTC is set correctly

- Check if battery backup is working
- After power loss

Format: `YYYY-MM-DD HH:MM:SS` (24-hour format)

`--settime [TIMESTAMP]`

Description: Set the RTC to a specific time or sync with PC time.

Usage:


bash

```
# Set to current PC time (easiest)
python datalogger_cli.py --settime now

# Set to specific time
python datalogger_cli.py --settime "2025-10-07 14:30:00"

# Without argument defaults to 'now'
python datalogger_cli.py --settime
```

Example Output:

```
✓ RTC set to: 2025-10-07 14:30:00
```

Important Notes:

- RTC will keep running even after power loss (if battery present)
- Use quotes for specific timestamps
- Format must be: `YYYY-MM-DD HH:MM:SS`
- Time is set in 24-hour format

Common Scenarios:


bash

```
# Initial setup
python datalogger_cli.py --settime now

# After battery replacement
python datalogger_cli.py --settime now

# Set to specific moment (e.g., experiment start)
python datalogger_cli.py --settime "2025-10-07 09:00:00"
```

Sensor Reading Commands

`--sense`

Description: Read current temperature, humidity, and battery voltage.

Usage:

bash

```
python datalogger_cli.py --sense
```

Example Output:

```
Temperature: 25.34°C
Humidity: 62.15%
Battery: 3712 mV (3.712 V)
```

Sensor Details:

- **Temperature:** SHT4x sensor, ±0.2°C accuracy
- **Humidity:** SHT4x sensor, ±2% RH accuracy
- **Battery:** Voltage divider measurement

When to use:

- Quick health check
- Verify sensors are working
- Monitor battery level
- Check environmental conditions

Note: Battery voltage shown only if firmware supports it.

`--vbat`

Description: Query battery voltage only (faster than `--sense`).

Usage:


bash

```
python datalogger_cli.py --vbat
```

Example Output:

```
Battery: 3712 mV (3.712 V)
```

Voltage Ranges:

- **4.2V:** Fully charged (LiPo/Li-ion)
- **3.7V:** Nominal voltage
- **3.3V:** Low battery warning
- **3.0V:** Critical (device may shutdown)

When to use:

- Regular battery monitoring
- Before field deployment
- Troubleshooting power issues

Configuration Commands

`--interval`

Description: Query the current data logging interval.

Usage:


bash

```
python datalogger_cli.py --interval
```

Example Output:

```
Logging interval: 300 seconds (5.0 minutes)
```

When to use:

- Verify configuration after setup
- Check settings before data collection
- Document experiment parameters

`--setinterval <SECONDS>`

Description: Set the data logging interval (time between measurements).

Usage:

bash

```
# Set to 1 minute
python datalogger_cli.py --setinterval 60

# Set to 5 minutes
python datalogger_cli.py --setinterval 300

# Set to 15 minutes
python datalogger_cli.py --setinterval 900

# Set to 1 hour
python datalogger_cli.py --setinterval 3600
```

Example Output:

```
✓ Interval set to: 300 seconds (5.0 minutes)
```

Constraints:

- **Minimum:** 30 seconds
- **Maximum:** 86400 seconds (24 hours)
- Values outside range are clamped by firmware

Common Intervals:

Interval	Seconds	Use Case
30 sec	30	High-resolution monitoring
1 min	60	Rapid changes
5 min	300	Standard logging
15 min	900	Long-term monitoring
30 min	1800	Slow processes
1 hour	3600	Daily patterns

Storage Calculation:

- Each row ≈ 50 bytes
- 1 day @ 5 min = 288 rows ≈ 14 KB/day
- 1 month @ 5 min ≈ 420 KB

When to use:

- Initial device setup
- Changing experiment requirements
- Balancing resolution vs. storage

LED Heartbeat Commands

The heartbeat LED provides visual feedback that the logger is alive and logging.

`--heartbeat`

Description: Query current heartbeat LED status and blink period.

Usage:

bash

```
python datalogger_cli.py --heartbeat
```

Example Output:

```
Heartbeat LED: ENABLED
Heartbeat period: 10 seconds
```

When to use:

- Check if LED is enabled
- Verify blink period setting
- Troubleshooting LED issues

`--heartbeat-on`

Description: Enable the heartbeat LED.

Usage:

bash

```
python datalogger_cli.py --heartbeat-on
```

Example Output:

```
✓ Heartbeat LED enabled
```

When to use:

- Field deployment (visual confirmation)
- Demonstrations
- Troubleshooting (verify device is alive)

Behavior:

- LED blinks periodically
- Period set by `--setheartbeat`
- Continues even during deep sleep

`--heartbeat-off`

Description: Disable the heartbeat LED.

Usage:

bash

```
python datalogger_cli.py --heartbeat-off
```

Example Output:

```
✓ Heartbeat LED disabled
```

When to use:

- Battery conservation (saves ~1-2 mA average)
- Light-sensitive experiments
- Stealth monitoring

`--setheartbeat <SECONDS>`

Description: Set the heartbeat LED blink period.

Usage:

bash

```
# Blink every 5 seconds
python datalogger_cli.py --setheartbeat 5

# Blink every 10 seconds
python datalogger_cli.py --setheartbeat 10

# Blink every 30 seconds
python datalogger_cli.py --setheartbeat 30

# Blink every minute
python datalogger_cli.py --setheartbeat 60
```

Example Output:

```
✓ Heartbeat period set to: 10 seconds
```

Recommended Periods:

- **5 seconds:** High visibility (uses more power)
- **10 seconds:** Balanced (default)
- **30 seconds:** Low power
- **60 seconds:** Ultra low power

Power Impact:

- Shorter period = more blinks = more power
- Each blink ≈ 20mA for ~100ms

Storage Management Commands

`--flash-info`

Description: Display complete flash partition table.

Usage:

bash

```
python datalogger_cli.py --flash-info
```

Example Output:

```
================================================================================
Internal Flash (all partitions)
================================================================================
Type/Sub       Label       Address    Size          Encrypted
--------------------------------------------------------------------------------
APP  /factory  factory     0x010000   1.00 MB       no
DATA /nvs      nvs         0x009000   24.00 KB      no
DATA /phy      phy_init    0x00F000   4.00 KB       no
DATA /littlefs spiffs      0x110000   2.88 MB       no
```

Information Shown:

- **Type:** APP (program) or DATA (storage)
- **Subtype:** Partition purpose (factory, nvs, littlefs, etc.)
- **Label:** Partition name
- **Address:** Flash memory location
- **Size:** Partition capacity
- **Encrypted:** Whether flash encryption is enabled

When to use:

- Understanding device storage layout
- Verifying partition sizes
- Troubleshooting storage issues
- Technical documentation

`--flash-free`

Description: Quick summary of LittleFS storage usage.

Usage:

bash

```
python datalogger_cli.py --flash-free
```

Example Output:

```
==========================================================================================
LittleFS (quick usage)
==========================================================================================
Mount    : OK
Total    : 2949120 (2.81 MB)
Used     : 421888 (412.00 KB)
Free     : 2527232 (2.41 MB)
Free %   : 85.69%
```

When to use:

- Quick storage check
- Before logging long experiments
- Deciding when to download/delete data

Storage Guidelines:

- **>50% free:** Plenty of space
- **20-50% free:** Consider downloading data
- **<20% free:** Download and delete old logs
- **<10% free:** Critical, may stop logging

`--flash-verbose`

Description: Detailed LittleFS partition and usage information.

Usage:


bash

```
python datalogger_cli.py --flash-verbose
```

Example Output:

```
================================================================================
Partition (LittleFS on internal flash)
================================================================================
Label     : spiffs
Type/Sub  : DATA / littlefs
Address   : 0x110000
Size      : 2949120 (2.81 MB)
Encrypted : no

================================================================================
LittleFS (usage)
================================================================================
Mount     : OK
Total     : 2949120 (2.81 MB)
Used      : 421888 (412.00 KB)
Free      : 2527232 (2.41 MB)
Free %    : 85.69%
```

When to use:

- Comprehensive storage analysis
- Technical troubleshooting
- Documentation and reporting

`--delete <YYYY-MM-DD>`

Description: Delete a specific date's log folder and all its files.

Usage:

bash

```
# Delete single date
python datalogger_cli.py --delete 2025-10-01

# Delete multiple dates (run separately)
python datalogger_cli.py --delete 2025-10-01
python datalogger_cli.py --delete 2025-10-02
python datalogger_cli.py --delete 2025-10-03
```

Example Output:

```
✓ Deleted 2025-10-01
  Files removed: 3
  Space freed: 127 KB (130,048 bytes)
```

What Gets Deleted:

- `/logs/YYYY-MM-DD/` folder
- CSV file inside
- Any other files in that date folder

Safety:

- Requires exact date match
- Non-reversible operation
- Download data before deleting

When to use:

- Free up storage space
- Remove old/unwanted data
- After backing up to PC

Best Practice:

bash

```
# 1. List available logs
python datalogger_cli.py --list

# 2. Download what you want to keep
python datalogger_cli.py --date 2025-10-01 --out ./backup/2025-10-01.csv

# 3. Verify download
ls -lh ./backup/2025-10-01.csv

# 4. Delete from device
python datalogger_cli.py --delete 2025-10-01
```

`--format <CODE>`

Description: Format (erase) entire LittleFS storage. **DESTRUCTIVE!**

Usage:

bash

```
# Default code is 246810 (check your firmware)
python datalogger_cli.py --format 246810
```

Example Output:

```
Formatting storage...
✓ Storage formatted
  Total: 2,949,120 bytes
  Used: 0 bytes
  Free: 2,949,120 bytes
```

⚠️ **WARNING:**

- **ALL DATA WILL BE LOST**
- Cannot be undone
- Requires security code
- Use only when necessary

Security:

- Default code: `246810` (set in firmware)
- Change code with `--setfmtcode`
- Max 3 attempts per session
- Code stored in NVS (survives format)

When to use:

- Device malfunction/corruption
- Factory reset
- Selling/transferring device
- File system errors

Before Formatting:

1. Download all important data
2. Verify downloads are complete
3. Have security code ready
4. Understand data will be lost

`--setfmtcode <OLD> <NEW>`

Description: Change the format security code.

Usage:

bash

```
# Change from default (246810) to custom code
python datalogger_cli.py --setfmtcode 246810 mynewcode123

# Change code again
python datalogger_cli.py --setfmtcode mynewcode123 anothernewcode
```

Example Output:

```
✓ Format security code changed
```

Requirements:

- Must provide correct old code
- New code: 4-32 characters
- Cannot reuse old code
- Authentication required

Security Best Practices:

- Use unique code per device
- Store code securely
- Don't share with unauthorized users
- Document code in safe location

When to use:

- Initial device setup
- Security policy compliance
- After unauthorized access attempt
- Transferring device ownership

System Control Commands

`--reboot`

Description: Restart the datalogger device.

Usage:

bash

```
python datalogger_cli.py --reboot
```

Example Output:

```
✓ Device rebooting... (reason: user)
```

What Happens:

1. Saves current state
2. Closes file system safely
3. Performs soft reset
4. Device starts fresh
5. Resumes logging automatically

When to use:

- After changing configuration
- Device seems unresponsive
- After firmware update
- Clear temporary errors

Safe to Reboot:

- Data is saved before reboot
- RTC keeps time (battery backup)
- Settings preserved in NVS
- Logging resumes automatically

Error Messages

Connection Errors

```
ERROR: Failed to open COM19: [Errno 2] could not open port
```

Cause: Port doesn't exist or is in use. **Solution:**

- Check Device Manager (Windows) or `ls /dev/tty*` (Linux)
- Close other programs using the port
- Try unplugging and replugging USB

```
ERROR: Could not auto-discover device. Use --port.
```

Cause: No device found on any port. **Solution:**

- Manually specify port: `--port COM19`
- Check USB connection
- Verify device is powered on
- Try `--reset` flag

```
Warning: No 'OK' to HELLO; continuing anyway.
```

Cause: Device didn't respond to handshake. **Solution:**

- Usually safe to ignore
- Try `--reset` if commands fail
- Check baud rate (should be 115200)

Command Errors

```
ERR code=RTC msg="not found"
```

Cause: RTC hardware not detected. **Solution:**

- Check I2C wiring (SDA, SCL)
- Verify RTC is powered
- Check firmware I2C configuration

```
ERR code=SHT4X msg="not found"
```

Cause: Temperature sensor not detected. **Solution:**

- Check sensor wiring
- Verify sensor power
- Test with `--sense` repeatedly

```
ERR code=BAD_DATE msg="format YYYY-MM-DD"
```

Cause: Invalid date format. **Solution:**

- Use exact format: `2025-10-07`
- Check year (2000-2099)
- No extra spaces

```
ERR code=NOT_FOUND msg="no file for 2025-10-07"
```

Cause: No log exists for that date. **Solution:**

- Use `--list` to see available dates
- Check spelling
- Verify device was logging on that date

```
ERR code=AUTH msg="bad_code"
```

Cause: Incorrect format security code. **Solution:**

- Check default code (usually `246810`)
- Verify code hasn't been changed
- Contact administrator if lost

```
ERR code=LOCKED msg="too many attempts"
```

Cause: Multiple failed format attempts. **Solution:**

- Reboot device: `--reboot`
- Try again with correct code
- Wait for session timeout

```
ERR code=NVS msg="write failed"
```

Cause: Non-volatile storage write error. **Solution:**

- Try command again
- Reboot device
- Check for corruption (rare)

Download Errors

```
CRC mismatch: got 12345678, expected 87654321
```

Cause: Data corruption during transfer. **Solution:**

- Try download again
- Check USB cable quality
- Use shorter/better cable
- Reduce interference

```
Timeout waiting for DATA
```

Cause: Device didn't send file data. **Solution:**

- File might not exist (check `--list`)
- Try again
- Reboot device if persistent

Troubleshooting

Device Not Responding

Symptoms:

- Commands timeout
- No response to HELLO
- Connection established but no data

Solutions:

1. Try hard reset:


bash

```
python datalogger_cli.py --reset --port COM19 --time
```

2. Manually reboot device:


bash

```
python datalogger_cli.py --reboot
```

3. Check power:


bash

```
python datalogger_cli.py --vbat
```

If battery < 3.0V, charge/replace

4. Verify port and baud:


bash

```
python datalogger_cli.py --port COM19 --baud 115200 --list
```

Slow Downloads

Symptoms:

- Download takes several minutes
- Progress bar moves slowly

Causes:

- Large files (normal)
- USB interference
- Poor cable

Solutions:

1. Use `--no-progress` for cleaner output:

bash

```
python datalogger_cli.py --date 2025-10-07 --no-progress
```

2. Use shorter/better USB cable
3. Reduce USB hub hops (direct connection)
4. Check file size first:

bash

```
python datalogger_cli.py --list
```

Storage Full

Symptoms:

- `Free %` near 0%
- Cannot download newer logs
- Device stops logging

Solutions:

1. Download all data:

bash

```
python datalogger_cli.py --list
# Download each date
```

2. Delete old logs:

bash

```
python datalogger_cli.py --delete 2025-09-01
python datalogger_cli.py --delete 2025-09-02
```

3. Check free space:


bash

```
python datalogger_cli.py --flash-free
```

4. Last resort - format (loses all data):


bash

```
python datalogger_cli.py --format 246810
```

RTC Losing Time

Symptoms:

- Time resets after power loss
- Time drift over days
- Wrong timestamps in logs

Solutions:

1. Set time:


bash

```
python datalogger_cli.py --settime now
```

2. Check RTC battery (if removable)
3. Verify RTC responds:


bash

```
python datalogger_cli.py --time
```

4. If persistent, may be hardware issue

Permission Denied (Linux/Mac)

Symptoms:

```
ERROR: [Errno 13] Permission denied: '/dev/ttyUSB0'
```

Solutions:

1. Add user to dialout group:

bash
```
sudo usermod -a -G dialout $USER
# Log out and back in
```

2. Temporary fix (not recommended):

bash
```
sudo python3 datalogger_cli.py --port /dev/ttyUSB0 --list
```

3. Check port ownership:

bash
```
ls -l /dev/ttyUSB0
```

Quick Reference Card

Essential Commands

bash

```
# Connection
--port COM19              # Specify port
--discover                # Auto-find device (default)
--reset                   # Reset before connecting

# Data
--list                    # Show available logs
--date 2025-10-07         # Download specific date
--out ./file.csv          # Custom output path

# RTC
--time                    # Show current time
--settime now             # Set to PC time

# Sensors
--sense                   # Temp, humidity, battery
--vbat                    # Battery voltage only

# Config
--interval                # Show log interval
--setinterval 300         # Set to 5 minutes

# LED
--heartbeat               # Show LED status
--heartbeat-on            # Enable LED
--setheartbeat 10         # Set 10 sec period

# Storage
--flash-free              # Show storage space
--delete 2025-10-01       # Delete date logs
--format 246810           # Format storage (DANGER!)

# System
--reboot                  # Restart device
```

Common Workflows

Initial Setup:

bash

```
python datalogger_cli.py --settime now
python datalogger_cli.py --setinterval 300
python datalogger_cli.py --heartbeat-on
python datalogger_cli.py --sense
```

Daily Check:


bash

```
python datalogger_cli.py --time
python datalogger_cli.py --vbat
python datalogger_cli.py --flash-free
python datalogger_cli.py --list
```

Data Retrieval:


bash

```
python datalogger_cli.py --list
python datalogger_cli.py --date 2025-10-07 --out ./data.csv
```

Maintenance:


bash

```
python datalogger_cli.py --list
python datalogger_cli.py --delete 2025-09-01
python datalogger_cli.py --flash-free
```

Support & Documentation

For additional help:

- Run `python datalogger_cli.py -h` for command list
- Check firmware documentation for hardware details
- Verify firmware version supports all commands

- Contact technical support for hardware issues

Document Version: 1.0
Last Updated: October 2025
Compatible Firmware: v1.x with all serial commands